

Parser Implementation: Tradeoffs and Decisions

COOL Compiler — frontend/parser — top-down vs. bottom-up

The decision

The parser is implemented as a **hand-written recursive descent parser**, i.e. a **top-down, LL(1)** parsing strategy (with two local spots using one extra token of lookahead, resolved directly in code rather than through a grammar transformation). Bottom-up parsing (LR(0), SLR(1), LALR(1), canonical LR(1)), and any generated-parser tooling such as bison/yacc, was considered and deliberately ruled out for this phase.

How the two families actually differ, mechanically

Top-down (LL). Parsing starts at the grammar's start symbol and works downward, predicting which production to expand next from the upcoming token(s). A recursive descent parser is the direct hand-written form of this: one function per grammar rule, each function recognizing its own production and calling other functions for its sub-nonterminals.

Bottom-up (LR family). Parsing starts at the leaves (the tokens actually read) and works upward, repeatedly recognizing a right-hand side already on the stack and reducing it to the corresponding left-hand side, until the whole input reduces to the start symbol. This is driven by a shift/reduce/goto table computed in advance from the grammar — constructing LR item sets, closures, gotos, and (for SLR/LALR/LR(1)) lookahead sets.

This mechanism difference is the root of every reason below.

Why top-down, specifically for this project

1. Consistency with how the rest of this compiler is built

A shift/reduce table for anything beyond a toy grammar is not something a person derives by hand — it is computed algorithmically, and in practice nobody hand-builds an LR table directly; tools like bison/yacc exist because that table construction is itself a small compiler. Recursive descent has no equivalent derived artifact: every grammar rule maps directly, by inspection, to one C function. The lexer in this project was hand-rolled character by character rather than generated with flex, specifically so every mechanism could be understood and debugged directly. A bottom-up parser would be the first major break from that approach — not because the language demands it, but because the technique itself requires generated tooling to be practical at any real scale.

2. COOL's grammar does not contain the ambiguities LR parsing exists to resolve

The textbook case for preferring LR over LL is a grammar with genuine local ambiguity — the canonical example is the dangling-else problem, where `if (a) if (b) x; else y;` is structurally ambiguous and needs either a precedence declaration or a grammar rewrite to resolve. COOL does not have this category of problem: every compound construct is explicitly closed (`if...then...else...fi`, `while...loop...pool`, `case...of...esac`), so there is never a point where the parser has ambiguous information about which enclosing construct an upcoming token belongs to. The category of problem that makes bottom-up parsing earn its complexity premium in real languages such as C simply does not exist in COOL's grammar as written.

3. Traceability and debuggability

A recursive descent parser's call stack mirrors the grammar directly: a trace through `parse_if_expr → parse_expr → parse_add_expr` is a readable account of exactly which rule is being recognized. An LR parser's runtime state is a stack of opaque, generated state numbers and table lookups — diagnosing a bug means cross-referencing the generated table back to the grammar, not reading code that resembles the grammar itself. This matches the same debugging-first reasoning already applied throughout the lexer's design (the Scanner cursor abstraction, the per-category `scan_*` functions).

Situations that would favor bottom-up parsing instead

To be fair to the alternative rather than dismiss it outright, bottom-up parsing has one genuine structural advantage worth stating plainly: **LR parsers handle left recursion naturally, for free**. A rule like `expr → expr '+' term` is not a problem for a shift/reduce mechanism the way it is for naive top-down parsing, which must either reject it outright or be rewritten into an iterative form first.

Bottom-up (LR-family) parsing would have been the better-justified choice if any of the following held for this project:

- The grammar were **large and organically evolving** rather than small and fixed, so that the fixed upfront cost of building or configuring LR tooling would be amortized over many future grammar changes.
- The language contained **genuine structural ambiguity** — constructs without an explicit closing keyword, where the correct parse depends on a precedence/associativity declaration rather than being resolvable by the grammar's shape alone.
- The grammar were **heavily and unavoidably left-recursive** in ways that could not be cleanly rewritten as iteration (precedence-climbing loops), making the LL rewrite itself the more complex path rather than the simpler one.
- A **generated-parser toolchain (bison/yacc) were already the target**, e.g. to match the reference COOL compiler's own implementation approach, or where parser-generator output was itself a project requirement rather than hand-written code.
- The project's goal were **maximum grammar coverage with minimal hand-written disambiguation logic**, accepting the opacity of generated tables as a reasonable tradeoff for not having to manually resolve lookahead conflicts in code.

None of these conditions hold for this project at this phase: the grammar is small and under full control, COOL's constructs are already explicitly disambiguated by design, left recursion is limited to the handful of

expression-precedence tiers (cleanly rewritten as iteration below), and the project's explicit goal has been hand-built, inspectable mechanisms throughout.

Efficiency and speed — why this was not the deciding factor

Both approaches parse in linear time, $O(n)$ in the number of tokens, for any grammar that is actually LL(1) or LALR(1) — which COOL's grammar is, under either lens. Neither family is asymptotically faster than the other. The real-world constant-factor difference (a table lookup vs. a function call and a few comparisons) is on the order of nanoseconds per token, and is irrelevant at the scale this compiler operates at: a few hundred to a few thousand tokens per source file, where disk I/O to read the file dominates either parser's runtime by orders of magnitude.

Where a real difference does exist, it favors recursive descent here: **build-time and maintenance cost**, not runtime speed. There is no table-generation step, no separate build artifact to regenerate whenever the grammar changes, and no generated-source file to keep in sync with hand-written semantic actions.

The deciding factor, stated plainly

The decision never actually turned on speed — both are fast enough that it is a non-factor for this project's scale. It turned on **implementation cost versus grammar complexity**: bottom-up parsing's cost (building table-generation machinery) is roughly fixed regardless of the grammar, while its benefit (gracefully handling genuinely ambiguous or heavily left-recursive grammars) scales with how messy the grammar actually is. COOL's grammar, kept small and already disambiguated by its own explicit closing-keyword design, does not have enough of that messiness to justify the fixed cost. Recursive descent, consistent with every other hand-built mechanism in this project so far, is the better-justified choice for this specific language at this phase — not a default reached without comparison.

Known, accepted costs of this choice

- Left recursion in the grammar must be eliminated by hand, via rewriting each precedence tier as an iterative loop (precedence climbing) rather than a textbook ϵ -production transform — addressed directly in the accompanying grammar document.
- Strict LL(1) requires FIRST-set disjointness between alternative productions of the same nonterminal; two spots in COOL's grammar (assignment vs. a bare expression starting with ID, and self-dispatch vs. a bare object reference, both starting with ID) are resolved with one extra token of lookahead inside the relevant function, rather than by a formal grammar transformation — a flexibility available to a hand-written parser that a generated LL(1) table would not permit without an explicit left-factoring step.
- Error recovery (skipping to a synchronizing token after a parse error) must be implemented explicitly, by hand, following the same non-fatal, collect-and-continue philosophy already used for lexical errors — a generated parser would not provide this for free either.